TP5 : Les fonctions

Consignes :

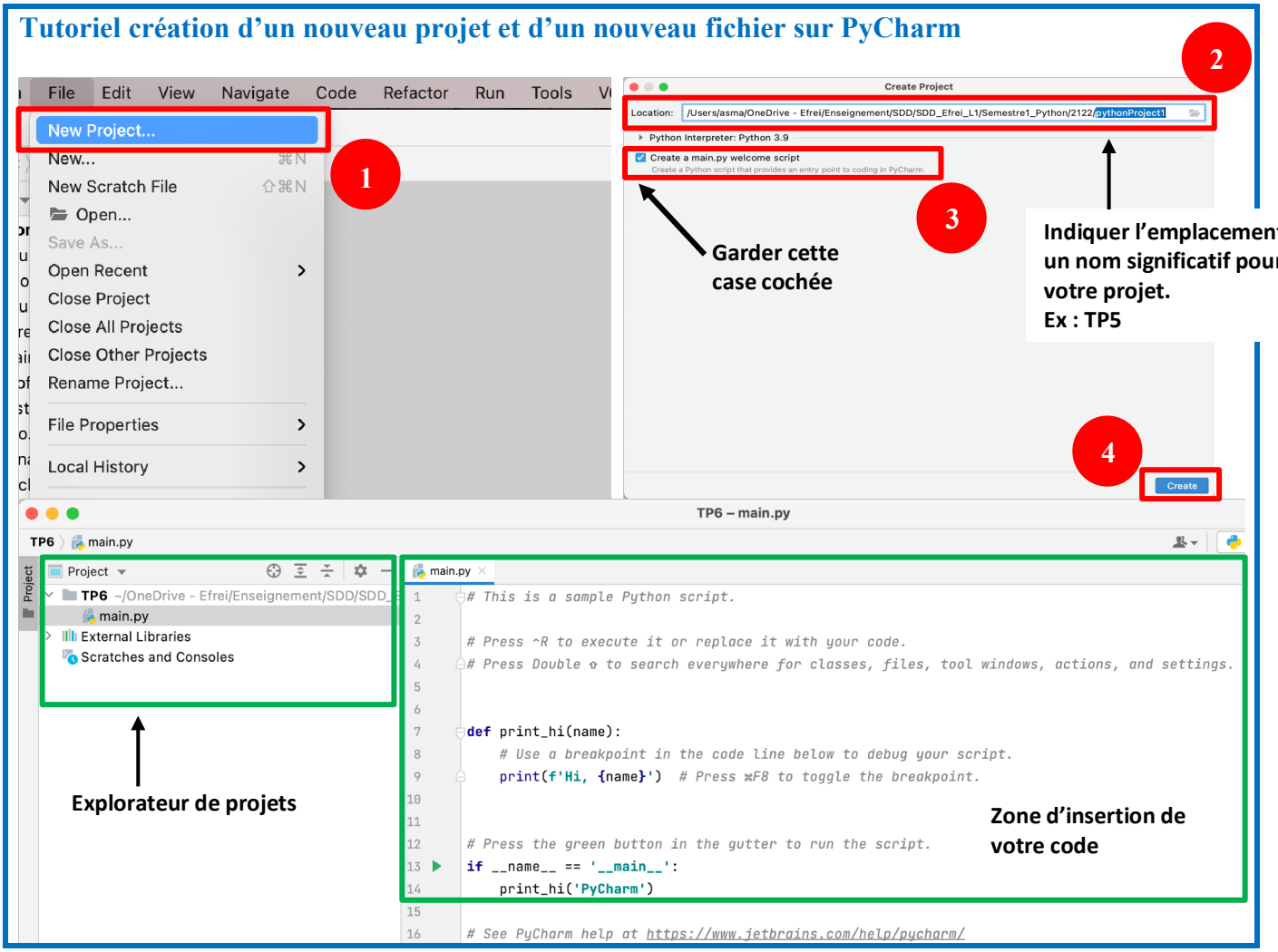
- Ce travail est INDIVIDUEL
- L'IDE à utiliser est **PyCharm**
- Il est fortement recommandé d'ouvrir le support du **CM 4** durant toute la séance du **TP5**
- Toutes les fonctions sont à écrire dans un fichier séparés appelé **functions.py**
- Tous les tests des fonctions seront à faire dans le fichier **main.py**
- Les instructions print() et input() sont interdites dans les fonctions sauf si le rôle même de la fonction est de permettre l'affichage ou la saisie d'informations utilisateur.
- Bien suivre les étapes indiquées dans le tutoriel pour la création de votre projet.

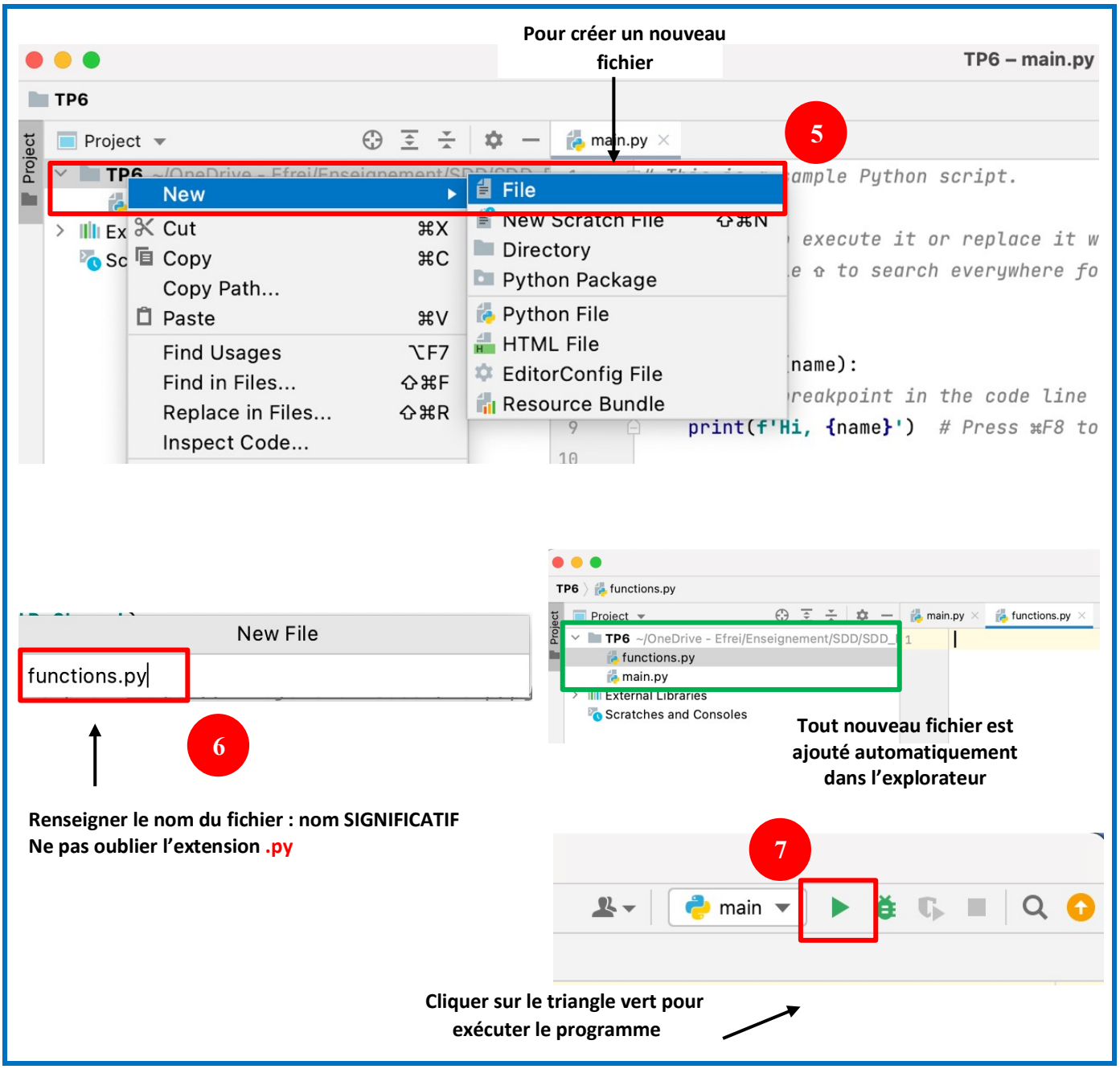
Acquis d'apprentissage :

A la fin de ce TP, chaque étudiant doit être capable de :

- Créer un projet sur PyCharm
- Créer un nouveau fichier
- Importer un module Python ou un module utilisateur
- Exécuter un programme sur PyCharm
- Écrire une fonction en langage Python
- Appeler une fonction en langage Python.

Tutoriel création d'un nouveau projet et d'un nouveau fichier sur PyCharm





Niveau débutant

Exercice 1

Que vont afficher les programmes suivants ? Corriger si nécessaire. :

a.
```
def carre(x):
    y = x**2
    return y
# Programme principal testant la fonction Pair
if __name__ == '__main__':
    carre(5)
```

b.
```
def add_list(l, val):
    l.append(val)

# Programme principal testant la fonction
if __name__ == '__main__':
    L = [1, 2, 3]
    add_list(L)
    print(L)
```

c.
```
def afficher_val(inf, sup):
    for i in range(inf, sup + 1):
        print(i)

# Programme principal testant la fonction Pair
if __name__ == '__main__':
        print(afficher_val(3, 10))
```

Exercice 2

Transformer le programme Python qui suit en une fonction qui sera appelée à partir d'un programme principal.

```
deb = int(input("Saisir un entier : "))
fin = int(input("Saisir un entier : "))
while fin < deb:
    fin = int(input("Saisir un entier : "))

pas = int(input("Saisir un entier : "))
i = deb
while i < fin:
    print(i)
    i = i + pas
```

Exercice 3

Compléter le code suivant afin de déterminer si un nombre est pair ou impair :

```
# Fonction qui teste si un entier est pair ou pas
def pair(…):
    if …. == 0:
        return …
    else:
        …
# Programme principal testant la fonction Pair
if __name__ == '__main__':
    val = int(input("Saisir un entier: "))
    if …:
        print("{} est un nombre … ".format(…))
    else:
        …
```

Pour toute la partie qui suit, suivez le modèle d'organisation indiqué dans les fichiers type main.py et functions.py fourni en annexe de ce TP

Niveau intermédiaire (Sur PyCharm)

Exercice 1

1. Écrire en langage Python une fonction qui prend en paramètres 3 entiers et qui retourne en sortie le maximum des 3 valeurs.
2. Dans le programme principal, demandez à l'utilisateur de saisir 3 entiers et testez votre fonction.

Exercice 2

1. Écrire en langage Python une fonction qui prend en paramètres 1 entier et qui retourne **True** s'il est premier, **False** sinon.
2. Dans le programme principal, demandez à l'utilisateur de saisir 1 entier et testez votre fonction.

Exercice 3

1. Écrire en langage Python une fonction qui prend en paramètres une liste 1D d'entiers et qui retourne la somme de toutes ses valeurs.
2. Dans le programme principal, initialisez une liste en dur et testez votre fonction.

Exercice 4

1. Écrire en langage Python une fonction qui prend en paramètres une liste 1D d'entiers ainsi qu'une valeur val et qui retourne la position de la première occurrence de val si elle existe dans la liste, -1 sinon.
2. Dans le programme principal, initialisez une liste en dur, demandez à l'utilisateur de saisir une valeur et testez votre fonction.

Exercice 5

1. Écrire en Python une fonction qui prend en paramètres une liste 1D d'entiers et renvoie une nouvelle liste contenant les éléments uniques de la première liste.

   Exemple :

   - Liste d'exemple : [1, 2, 3, 3, 3, 3, 4, 5]
   - Liste unique : [1, 2, 3, 4, 5]
2. Dans le programme principal, initialiser une liste 1D et tester votre fonction.

Exercice 6

1. Écrire en langage Python une fonction qui prend en paramètres une chaîne de caractères et qui retourne son inverse.
2. Dans le programme principal, demandez à l'utilisateur de saisir une chaîne de caractères et de tester votre fonction.

Exercice 7

1. Écrire en langage Python une fonction qui prend en paramètres une chaîne de caractères et qui retourne le nombre de voyelles qu'elle contient.
2. Dans le programme principal, demandez à l'utilisateur de saisir une chaîne de caractères et de tester votre fonction.

Exercice 8

1. Écrire en langage Python une fonction qui prend en paramètres deux fichiers f1 et f2 et qui permet d'ajouter le contenu de f1 à la fin du contenu de f2
2. Dans le programme principal, testez votre fonction.

Exercice 9

1. Écrire en langage Python une fonction qui prend en paramètres une liste à 2D et qui remplace les valeurs de toutes les lignes de rangs pairs par leur double.

   Exemple :

   Matrice initiale :

```
M = [[0, 4, 8, 12, 1, 0],
     [1, 4, 5, 3, 9, 8],
     [1, 0, 1, 2, 6, 9],
     [1, 1, 5, 5, 3, 8],
     [0, 0, 3, 2, 1, 8]
     ]
```

   Matrice après l'appel de la fonction :

```
M = [[0, 8, 16, 24, 2, 0],
     [1, 4, 5, 3, 9, 8],
     [2, 0, 2, 4, 12, 18],
     [1, 1, 5, 5, 3, 8],
     [0, 0, 6, 4, 2, 16]
     ]
```

2. Dans le programme principal, initialisez une matrice en dur et testez votre fonction.

Exercice 10

1. Écrire en langage Python une fonction qui prend en paramètres deux listes L1 et L2 et qui retourne une liste L3 où les éléments de L1 et L2 sont insérés alternativement.

   Exemple :

```
L1 = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
L2 = ['Janvier', 'Février', 'Mars', 'Avril', 'Mai', 'Juin', 'Juillet', 'Août',
'Septembre', 'Octobre', 'Novembre', 'Décembre']
L3 = ['Janvier', 31, 'Février', 28, 'Mars', 31, 'Avril', 30, 'Mai', 31,
'Juin', 30, 'Juillet', 31, 'Août', 31, 'Septembre', 30, 'Octobre', 31,
'Novembre', 30, 'Décembre', 31]
```

2. Dans le programme principal, initialisez deux listes, appelez votre fonction et afficher le résultat.

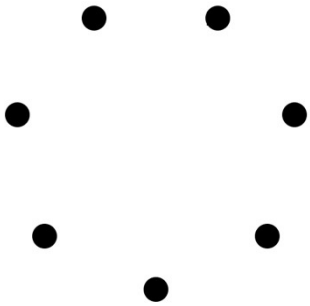
Exercice 11

1. Écrire en langage Python une fonction **draw_dots** qui prend en paramètres un entier et qui dessine à l'écran n points de façon circulaire.

   Indication : Utiliser la fonction `dot()` du module Turtle.

   Exemple :

   - L'exécution de la fonction dot(20) affiche le point suivant -
     Si n = 7, afficher à l'écran :



2. Dans le programme principal, demandez à l'utilisateur de saisir une valeur et lui afficher la figure qui lui correspond.

Niveau avancé (Sur PyCharm)

Exercice 1

1. Écrire en langage Python une fonction **fill_list** qui permet de remplir une liste de n valeurs réelles générées aléatoirement entre 0 et 1 (exclu) et qui sont arrondies à 2 chiffres après la virgule.
2. Écrire une fonction **average_list** qui prend en paramètres une liste et qui calcule la moyenne de ses valeurs.
3. Écrire une fonction **print_list** qui prend en paramètre une liste et qui permet d'afficher ses valeurs séparées par des espaces.
4. Dans le programme principal, appelez les fonctions précédentes afin :
   a. D'initialiser une liste vide de nb valeurs réelles (nb étant saisie par l'utilisateur)
   b. D'afficher le contenu de la liste remplie
   c. Calculer la moyenne de ses valeurs
   d. Afficher le résultat final à l'écran.

Exercice 2

1. Écrire en langage Python une fonction qui vérifie si une chaine de caractères est un pangramme. Un pangramme est une chaîne de caractères dans laquelle chaque lettre de l'alphabet apparaît au moins une fois.

Exemple de pangramme:

"The quick brown fox jumps over the lazy dog"

2. Dans le programme principal, testez votre fonction.

Exercice 3

1. Écrire une fonction qui reçoit en paramètres une séquence de mots séparés par un trait d'union et qui permet de les afficher sous forme d'une séquence de mots triés par ordre alphabétique et séparés par des traits d'union.

   Exemple:

   Entrée: green-red-yellow-black-white
   Sortie: black-green-red-white-yellow

2. Dans le programme principal, testez votre fonction.

Exercice 4

Écrire la fonction **volBoite(x1, x2,x3)** de manière à ce qu'elle puisse être appelée avec un, deux, ou trois arguments.

a. Avec 1 argument, la boîte est considérée comme cubique (l'argument étant l'arête de ce cube) et donc elle doit retourner le volume du cube (côté puissance 3).

b. Avec 2 arguments, la boîte est considérée comme un prisme à base carrée. Dans ce cas le premier argument est le côté du carré, et le second la hauteur du prisme. Ainsi, elle doit retourner le volume du prisme qui se calcule comme étant 2 x le côté du carré multiplié par la hauteur.

c. Avec 3 arguments, la boîte est considérée comme un parallélépipède et son volume est calculé comme étant le produit des 3 dimensions.

Exemple :

La fonction doit retourner les valeurs suivantes après ces tests :

```
print volBoite() # donne -1 (-> erreur) print
volBoite(5.2) # donne 140.608 print
volBoite(5.2, 3) # donne 81.12 print
volBoite(5.2, 3, 7.4) # donne 115.44
```

Exercice 5

1. Reprendre la fonction **reverse_str** écrite précédemment qui prend en paramètre une chaîne de caractères et qui retourne son inverse.
2. Écrire une fonction **compare_str** qui prend en paramètres deux chaînes et caractères et qui détermine si elles sont égales ou pas.
3. Écrire une fonction **verif_palindrom** qui prend en paramètre une chaîne de caractères et qui vérifie par appels aux deux fonctions précédentes si c'est un palindrome ou pas.

4. Dans le programme principal :
   a. Demander à l'utilisateur de saisir une chaîne de caractères
   b. Vérifier s'il s'agit d'un palindrome ou pas.
   c. Afficher le message correspondant à l'utilisateur.

Exercice 6

1. Écrire en langage Python une fonction **pos_max** qui prend en paramètre une liste 1D ainsi qu'une valeur **borne_sup** et qui retourne l'indice de la valeur maximum se trouvant entre les cases 0 et **borne_sup**.
2. Écrire une fonction **swap_list** qui prend en paramètres une liste et deux indices **i** et **j** et qui permet de permuter les valeurs se trouvant aux positions i et j dans la liste.
3. Écrire une fonction **sort_list** qui prend en paramètre une liste et qui permet de la trier en appelant les deux fonctions précédentes.

   Idée : **(Observez bien les étapes ci-dessous et déduire le fonctionnement)**

   - **Étape 1 :** Chercher la position de la valeur maximum qui se trouve entre les indices **0** et **len(L)-1** (L étant la liste à trier)

12	**58**	**23**	**9**	**22**

   - **Étape 2 :** Pousser la valeur maximum trouvée au fond de la liste en la permutant avec la valeur déjà existante à la position **len(L) – 1**

12	**22**	**23**	**9**	**58**

   - **Étape 3 :** Chercher la position de la valeur maximum qui se trouve entre les indices **0** et **len(L)-2**.

12	**22**	**23**	**9**	**58**

   - **Étape 4 :** Pousser la valeur maximum trouvée au fond de la liste en la permutant avec la valeur déjà existante à la position **len(L) – 2**

12	**22**	**9**	**23**	**58**

   - Répéter les étapes précédentes et déduire une condition d'arrêt.
4. Dans le programme principal :
   a. Initialisez une liste en dur
   b. Appelez la fonction qui permet de la trier
   c. En utilisant une de vos fonctions précédentes (des autres exercices), affichez le contenu de la liste avant et après le tri.